

Wydział Informatyki

Katedra Systemów Inteligentnych i Data Science

Inteligentne systemy przetwarzania danych

Kamil Czyżewski s22370

Kuba Szulc s23640

Konfigurowalne środowisko AutoML dla
finansowych potoków predykcyjnych w
turnieju Numerai

Praca magisterska

Promotor techniczny Filip Dziecioł

Promotor główny Grzegorz Wójcik

Warszawa, 2026

Faculty of Computer Science

Department of Intelligent Systems and Data Science

Kamil Czyżewski s22370

Kuba Szulc s23640

A Configurable AutoML Framework for Financial Prediction Pipelines in the Numerai Tournament

Master's thesis

Technical supervisor Filip Dziecioł

Lead supervisor Grzegorz Wójcik

Warsaw, 2026

Abstrakt

Praca przedstawia projekt i implementację AlphaPulse — konfigurowalnego środowiska AutoML do budowy, strojenia, walidacji i eksportu finansowych potoków predykcyjnych dla turnieju Numerai. System łączy deklaratywne konfiguracje YAML, routing cech, modułarny preprocessing, modele tabelaryczne, optymalizację hiperparametrów z użyciem Optuna lub Ray Tune, walidację uwzględniającą strukturę er, rejestr artefaktów, diagnostykę Weights & Biases oraz eksport przenośnej funkcji predykcyjnej.

Ewaluacja obejmuje pełnoskalowy benchmark przetwarzania 2,75 mln wierszy uczących oraz końcową serię HPO na danych Numerai v5.3 dla celu `target_ender_60`. W serii końcowej oceniono 27 kompletnych konfiguracji na deterministycznej próbie 274 627 wierszy. Protokół wykorzystuje 16 er separacji, 52 ery holdoutu oraz niezależny przedział 88 er z predykcją Meta Modelu. Najlepszy wynik holdoutowy uzyskała próba 11 z modelem LightGBM: oficjalny CORR Sharpe równy 1,044 przy średnim CORR 0,0374. Na niezależnym przedziale próba 22 osiągnęła najwyższy CORR Sharpe 0,425, natomiast próba 4 najwyższy MMC Sharpe 0,306; obie konfiguracje tworzą front Pareto jakości sygnału i jego unikalności. Rezultaty potwierdzają, że AlphaPulse realizuje reprodukowalny cykl od danych do porównywalnych artefaktów eksperymentalnych i dostarcza kandydatów do końcowego treningu produkcyjnego.

Słowa kluczowe: uczenie maszynowe, AutoML, Numerai, walidacja czasowa, optymalizacja hiperparametrów

Abstract

This thesis presents the design and implementation of AlphaPulse, a configurable AutoML environment for building, tuning, validating and exporting financial prediction pipelines for the Numerai tournament. The system combines declarative YAML configurations, feature routing, modular preprocessing, tabular models, hyperparameter optimisation with Optuna or Ray Tune, era-aware validation, artefact tracking, Weights & Biases diagnostics and export of a portable prediction function.

The evaluation comprises a full-scale processing benchmark with 2.75 million training rows and a final HPO study on Numerai v5.3 data for the `target_ender_60` target. The final study evaluates 27 complete configurations on a deterministic sample of 274 627 rows. Its protocol uses a 16-era gap, a 52-era holdout and an independent 88-era interval with aligned Meta Model predictions. Trial 11 with LightGBM achieved the best holdout result: an official CORR Sharpe of 1,044 and mean CORR of 0,0374. On the independent interval, trial 22 obtained the highest CORR Sharpe of 0.425, while trial 4 obtained the highest MMC Sharpe of 0.306; together they form the Pareto frontier between signal quality and uniqueness. The results show that AlphaPulse provides a reproducible workflow from data to comparable experimental artefacts and identifies candidates for final production training.

Keywords: machine learning, AutoML, Numerai, temporal validation, hyperparameter optimisation

Spis treści

1	Wstęp	5
1.1	Kontekst i motywacja	5
1.2	Problem badawczy	6
1.3	Cele pracy	6
1.4	Zakres i ograniczenia	7
1.5	Wkład autorów	8
1.6	Kryteria sukcesu	8
2	Podstawy problemu i kontekst systemu AlphaPulse	9
2.1	Specyfika turnieju Numerai	9
2.2	Charakterystyka danych finansowych	10
2.3	Metryki jakości i kryteria oceny	10
2.4	Ryzyko wycieku informacji i poprawna walidacja	11
2.5	AutoML, HPO i modele tabelaryczne	11
2.6	Luka projektowa i pozycjonowanie AlphaPulse	11
2.7	Zakres funkcjonalny AlphaPulse jako punkt odniesienia dalszych rozdziałów	12
2.8	Mapowanie wyzwań domenowych na decyzje projektowe	13
2.9	Wnioski	13
3	Architektura systemu AlphaPulse	14
3.1	Założenia architektoniczne	14
3.2	Widok warstwowy systemu	14
3.3	Przepływ wykonania end-to-end	17
3.4	Komponenty funkcjonalne i ich odpowiedzialności	17
3.5	Wzorce zapewniające jakość inżynierską	17
3.6	Ograniczenia architektury	18
3.7	Diagram architektury logicznej	18
3.8	Kontrakty komponentów	19
4	Implementacja systemu AlphaPulse	20
4.1	Organizacja repozytorium	20
4.2	Warstwa uruchomieniowa (CLI)	20
4.3	Konfiguracja eksperymentów (YAML)	21
4.4	Implementacja warstwy danych	22

4.5	Implementacja pipeline’u modelowego	23
4.6	Implementacja HPO	24
4.7	Implementacja AutoResearch	25
4.8	Eksport predykcji i artefakty końcowe	25
4.9	Zapewnienie jakości kodu	26
4.10	Przykładowa konfiguracja referencyjna	27
4.11	Przestrzeń przeszukiwania HPO	27
5	Metodyka badań	28
5.1	Pytania eksperymentalne	28
5.2	Dwie serie o rozłącznych rolach	28
5.3	Dane i podział czasowy badania końcowego	29
5.4	Protokół HPO	30
5.5	Przestrzeń konfiguracji	30
5.6	Metryki i reguła selekcji	31
5.7	Metody analizy	31
5.8	Reprodukowalność	31
5.9	Zagrożenia dla trafności	32
6	Wyniki eksperymentów	33
6.1	Weryfikacja pełnej skali	33
6.2	Kompletność końcowej serii HPO	33
6.3	Rozkład oficjalnych metryk	34
6.4	Rodziny modeli w ustalonym budżecie	34
6.5	Przebieg optymalizacji	35
6.6	Transfer między przedziałami czasowymi	37
6.7	Kandydaci końcowi	37
6.8	Synteza wyników	39
7	Dyskusja i zakończenie	40
7.1	Realizacja celu pracy	40
7.2	Interpretacja wyników modelowych	40
7.3	Odpowiedzi na pytania eksperymentalne	41
7.4	Znaczenie inżynierskie	41
7.5	Ograniczenia	42
7.6	Rekomendacja wdrożeniowa i dalszy rozwój	42
7.7	Wnioski końcowe	43
	Bibliografia	44

Rozdział 1

Wstęp

1.1 Kontekst i motywacja

Prognozowanie rynków finansowych należy do najtrudniejszych problemów uczenia maszynowego. Dane finansowe wyróżniają się niskim stosunkiem sygnału do szumu, niestacjonarnością, silną zależnością czasową między obserwacjami oraz strukturą grupową (ery — tygodniowe przekroje rynku), która wyklucza zastosowanie standardowych technik podziału danych. Jednocześnie skuteczność predykcji jest weryfikowana przez rzeczywisty rynek, co eliminuje ryzyko nadmiernego dopasowania do artefaktów zbioru testowego.

Turniej Numerai, uruchomiony w 2015 roku przez firmę Numerai Inc., stanowi unikalną platformę badawczą w obszarze finansowego uczenia maszynowego. Numerai dostarcza uczestnikom zaszyfrowane, zanonimizowane dane tabelaryczne opisujące spółki giełdowe i ich atrybuty fundamentalne, techniczne oraz rynkowe. Zadaniem uczestników jest wytrenowanie modeli predykujących przyszłe stopy zwrotu neutralne względem rynku (alpha) w horyzoncie 20 dni roboczych. Prognozy wszystkich uczestników są agregowane przez fundusz hedgingowy Numerai do jednego Meta Modelu, który steruje rzeczywistymi decyzjami inwestycyjnymi. Uczestnicy mogą zbierać tokeny NMR (kryptowaluta Numerai) na swoje modele, zdobywając nagrody za pozytywne wyniki lub tracąc środki w przypadku słabych predykcji [5].

Ta architektura tworzy naturalny problem badawczy: jak systematycznie eksperymentować z konfiguracjami modeli ML, aby maksymalizować jakość predykcji mierzoną metrykami takimi jak korelacja per-era i współczynnik Sharpe’a, przy jednoczesnym spełnieniu rygorystycznych wymagań dotyczących formatu submisji i unikania wycieku informacji czasowej?

Ogólne platformy AutoML automatyzują dobór modeli i hiperparametrów, lecz nie rozwiązują automatycznie wymagań specyficznych dla Numerai: podziałów według er, separacji czasowej zależnej od horyzontu celu, oficjalnych metryk konkursowych oraz eksportu przenośnej funkcji predykcyjnej [10]. AlphaPulse integruje te elementy w jednym przepływie pracy — od pobrania danych, przez definicję i ocenę eksperymentu, po walidację artefaktu końcowego.

1.2 Problem badawczy

Niniejsza praca podejmuje następujący problem badawczy:

Jak zaprojektować i zaimplementować framework, który umożliwi systematyczne i reprodukowalne eksperymentowanie z potokami uczenia maszynowego dla danych finansowych turnieju Numerai, redukując jednocześnie nakład pracy inżynierskiej wymagany do testowania nowych konfiguracji?

Problem ten dekomponuje się na następujące pytania szczegółowe:

1. Jakie abstrakcje komponentów (preprocessory, modele, strategie ensemble) są niezbędne do obsługi pełnego spektrum konfiguracji stosowanych w danych finansowych o strukturze szeregów czasowych?
2. Jak zapewnić poprawną walidację krzyżową uwzględniającą zależności temporalne i korelacje wewnątrz er, by uniknąć wycieku informacji?
3. W jakim stopniu automatyzacja poszukiwania hiperparametrów i konfiguracji modeli – w tym z użyciem agentowego modelu językowego – może zastąpić lub wspomóc manualne eksperymenty badacza?
4. Jak zintegrować wszystkie etapy przepływu pracy w spójny, konfigurowalny system zgodny z wymaganiami Numerai?

1.3 Cele pracy

Głównym celem pracy jest zaprojektowanie i zaimplementowanie frameworku AlphaPulse, który:

- Ujednolica pełny przepływ pracy konkursu Numerai — od pobierania danych po eksport submisji.
- Umożliwia deklaratywne definiowanie eksperymentów w formacie YAML, bez konieczności modyfikowania kodu źródłowego.
- Zapewnia poprawną walidację z uwzględnieniem struktury er, purging i embargo.
- Automatyzuje poszukiwanie optymalnych hiperparametrów i konfiguracji pipeline’u za pomocą Optuna (tryb `--local`) lub Ray Tune (tryb rozproszony), z obsługą `--resume` i bazą TrialDB.

- Wprowadza autonomiczną pętlę badawczą (AutoResearch) sterowaną dużym modelem językowym (LLM), która iteracyjnie proponuje i testuje modyfikacje konfiguracji.
- Raportuje oficjalnie zdefiniowane CORR i MMC oraz ich agregaty per-era.
- Eksportuje wyniki w postaci pliku `predict.pkl` zgodnego z wymaganiami API Numerai oraz wspiera tygodniowy cykl produkcyjny: inferencja na `live.parquet` i automatyczna submitacja predykcji.

Celami pomocniczymi są: ocena skuteczności podejścia agentowego w kontekście AutoML, analiza wpływu wyboru modeli i strategii ensemble na metryki jakości predykcji oraz porównanie różnych strategii walidacji krzyżowej dla danych finansowych z strukturą er.

1.4 Zakres i ograniczenia

Framework AlphaPulse jest przeznaczony dla turnieju Numerai Classic, a badanie końcowe wykorzystuje zestaw danych w wersji v5.3. Zakres pracy obejmuje:

- Modele tabelaryczne: gradient boosting, modele drzewiste, modele liniowe, fundamentalne modele tabelaryczne (TabPFN, TabPFN3, TabICL) oraz uczenie głębokie (TabularDL z architekturami FT-Transformer i MLP).
- Preprocessory: skalowanie, redukcja wymiarowości, selekcja cech, autoencoder, kompresja cech, selektor stabilności per-era (EraStableFeatureSelector) i regularyzacja poprzez szum gaussowski.
- Strategie ensemble: `single`, `weighted`, `stacking`; potoki wielogłowicowe (MultiHeadPipeline) i wielocelowe (MultiTargetPipeline).
- Optymalizację hiperparametrów (Optuna TPE w trybie lokalnym, Ray Tune w trybie rozproszonym) oraz system AutoResearch z agentem Claude.
- Panel EDA (Streamlit) do eksploracji danych i wizualizacji wyników HPO.

Praca nie obejmuje: analizy modeli sekwencyjnych, czyli na przykład LSTM czy Transformer, dla danych czasowych, strategii tradingowych ani mechanizmów stakowania tokenów NMR, a także integracji z turniejem Numerai Signals.

1.5 Wkład autorów

Wkład pracy obejmuje:

- projekt i implementację frameworku **AlphaPulse** w wersji 0.6.0 (Python 3.12+, repozytorium `main`) [1],
- deklaracyjny format eksperymentów (Experiment v1 YAML) z katalogiem cech (`features/catalog.py`),
- integrację HPO (Optuna/Ray Tune, TrialDB) oraz pętli agentowej AutoResearch,
- oficjalne diagnostyki CORR i MMC oraz diagnostykę W&B (SHAP, ekspozycja cech, wykresy per-era),
- ujednolicony pipeline od pobrania danych Numerai przez eksport `predict.pkl` do inferencji *live* i submisji tygodniowej.

1.6 Kryteria sukcesu

Praca uznaje się za spełnioną, jeśli:

1. framework umożliwia powtarzalne uruchomienia tej samej konfiguracji (seed globalny, artefakt `*_provenance.json`),
2. HPO poprawnie realizuje próbkowanie, ocenę, persystencję i wznowienie prób,
3. AutoResearch poprawnie proponuje mutacje oraz zapisuje stan umożliwiający wznowienie,
4. końcowy artefakt submisji przechodzi walidację formatu i test dymny (*export smoke test*).
5. seria eksperymentalna identyfikuje co najmniej jedną konfigurację o dodatnim CORR Sharpe na holdoutcie i dodatnim MMC Sharpe na niezależnym przedziale walidacyjnym.

Rozdział 2

Podstawy problemu i kontekst systemu AlphaPulse

Celem niniejszego rozdziału jest przedstawienie kontekstu domenowego i technicznego, w którym powstał framework *AlphaPulse*. Rozdział porządkuje pojęcia wymagane do zrozumienia dalszych części pracy: specyfikę turnieju Numerai, charakter danych, kryteria oceny modeli, ograniczenia metodologiczne oraz wymagania, jakie musi spełniać nowoczesny system AutoML dla finansowych potoków predykcyjnych.

2.1 Specyfika turnieju Numerai

Numerai jest konkursem data science, w którym uczestnicy budują modele predykcyjne na zanonimizowanych danych tabelarycznych i dostarczają predykcje do wspólnego meta-modelu funduszu. W przeciwieństwie do klasycznych konkursów ML, nacisk jest położony nie tylko na jakość predykcji, ale również na stabilność sygnału i odporność na zmianę warunków rynkowych [2].

W praktyce oznacza to, że:

- dane mają strukturę czasową i grupową (ery),
- błędna walidacja może prowadzić do sztucznie zawyżonych wyników,
- wysokie znaczenie ma reprodukowalność eksperymentów,
- końcowym artefaktem jest plik predykcji zgodny z wymaganym formatem submisji.

Takie środowisko premiuje podejście systemowe: powtarzalne pipeline'y, kontrolę konfiguracji, jasne logowanie eksperymentów i automatyzację przeszukiwania przestrzeni modeli.

2.2 Charakterystyka danych finansowych

Dane używane w zadaniach Numerai należą do kategorii danych tabelarycznych o silnym komponencie czasowym. Z perspektywy modelowania najważniejsze cechy tych danych to:

- niski stosunek sygnału do szumu,
- niestacjonarność rozkładów cech i celu,
- zależności temporalne między obserwacjami,
- podatność na przeciek informacji przy niepoprawnym podziale danych.

Konsekwencją jest konieczność ostrożnego projektowania zarówno preprocesingu, jak i procedury walidacyjnej. W praktyce szczególnie istotne stają się metody walidacji uwzględniające strukturę er oraz separację czasową między zbiorem uczącym i walidacyjnym.

2.3 Metryki jakości i kryteria oceny

W odróżnieniu od wielu zadań regresyjnych, ocena modeli finansowych nie powinna opierać się wyłącznie na błędzie punktowym. Dla każdej ery e oficjalny CORR porównuje uporządkowane predykcje $\hat{y}_e$ z celem y_e po transformacjach określonych w dokumentacji Numerai [3]. Stabilność sygnału agreguje współczynnik Sharpe’a:

$$S_{\text{CORR}} = \frac{\bar{c}}{s_c}, \quad c = (c_1, \dots, c_E), \quad (2.1)$$

gdzie c_e oznacza oficjalny CORR w erze, $\bar{c}$ jego średnią, a s_c odchylenie standardowe między erami. AlphaPulse raportuje ponadto:

- średni CORR i odchylenie standardowe między erami,
- udział er z dodatnim CORR oraz maksymalne obsunięcie skumulowanego CORR,
- oficjalny MMC, który mierzy przyrost informacji po ortogonalizacji predykcji względem Meta Modelu, wraz z jego średnią i współczynnikiem Sharpe’a [4].

Implementacje zweryfikowano względem zamrożonej wersji `numerai-tools` 0.6.0 [6]. W badaniu głównym kryterium optymalizacji stanowi S_{CORR} na purged holdoutcie. CORR i MMC na niezależnym przedziale walidacyjnym służą do oceny transferu w czasie oraz wyboru kompromisu między siłą i unikalnością sygnału.

2.4 Ryzyko wycieku informacji i poprawna walidacja

Jednym z najważniejszych problemów metodologicznych w modelach finansowych jest *look-ahead bias*, czyli niejawne wykorzystanie informacji, która w momencie predykcji nie byłaby dostępna. Zjawisko to może wystąpić na wielu etapach:

- podczas przygotowania cech,
- przy podziale danych na train/validation,
- przy doborze hiperparametrów.

Dlatego system klasy *AlphaPulse* musi wspierać walidację zgodną z naturą danych czasowych i erowych, w tym podejścia *era-aware* oraz *purged cross-validation*. Wymóg ten stanowi fundament wiarygodnej oceny i bezpośrednio wpływa na konstrukcję architektury frameworku [7].

2.5 AutoML, HPO i modele tabelaryczne

Systemy AutoML automatyzują dobór algorytmu, transformacji i hiperparametrów, ale ich skuteczność zależy od poprawnej definicji przestrzeni poszukiwań i protokołu walidacji [10]. AlphaPulse wykorzystuje Optuna TPE do sekwencyjnego wyboru prób oraz Ray Tune do wykonania rozproszonego [8, 9]. Warstwa optymalizacji nie zastępuje metodyki badawczej: każda konfiguracja przechodzi ten sam podział czasowy, ten sam scoring i zapis proveniencji.

Przestrzeń modeli obejmuje algorytmy gradient boosting — XGBoost, LightGBM i CatBoost — które pozostają silnymi punktami odniesienia dla danych tabelarycznych [11, 12, 13]. Uzupełniają je modele liniowe, zespoły drzew oraz fundamentalne modele tabelaryczne, w tym TabPFN [15]. Z perspektywy badawczej ważna jest wspólna abstrakcja modelu: umożliwia porównanie rodzin w jednym protokole, bez wiązania warstwy danych i ewaluacji z konkretną biblioteką.

2.6 Luka projektowa i pozycjonowanie AlphaPulse

Ogólny framework AutoML zazwyczaj kończy działanie na wytrenowanym estymatorze. Tracker eksperymentów rejestruje przebieg, lecz nie definiuje po-

działu czasowego ani formatu wdrożenia. AlphaPulse łączy te role z wymaganiami Numerai, co przedstawia tabela 2.1.

Tabela 2.1: Pozycjonowanie AlphaPulse względem klas narzędzi wspierających eksperymenty ML

Klasa rozwiązania	Automatyzacja konfiguracji	Walidacja erowa	Eksport Numerai
Ogólny AutoML	wysoka	wymaga konfiguracji domenowej	brak
Tracker eksperymentów	nie	nie	brak
Skrypt konkursowy	zależna od implementacji	możliwa	zwykle pojedynczy format
AlphaPulse	HPO i AutoResearch	purge, holdout i walk-forward	predict.pkl i test dymny

Podójście *configuration-driven* rozdziela opis eksperymentu od mechaniki wykonania. Dzięki temu konfiguracja jest wersjonowalna, a ten sam graf potoku można uruchomić w trybie pojedynczym, HPO, diagnostycznym i eksportowym.

2.7 Zakres funkcjonalny AlphaPulse jako punkt odniesienia dalszych rozdziałów

Na poziomie operacyjnym framework obejmuje pełny przepływ pracy konkursowej:

1. pobranie danych (`train`, `validation`, `live`, `meta_model`),
2. uruchamianie eksperymentów opisanych YAML,
3. automatyczną optymalizację hiperparametrów (HPO),
4. agentową pętlę badawczą (*AutoResearch*),
5. eksport predykcji do formatu submisji,
6. inferencję na danych *live* i automatyczną submisję do API Numerai.

W projekcie uwzględniono również:

- modułarny preprocessing (m.in. skalowanie, PCA, autoenkoder, kompresja, selekcja stabilności per-era),

- szeroką rodzinę modeli tabelarycznych (boosting, modele drzewiaste, liniowe, TabPFN/TabPFN3/TabICL, TabularDL),
- potoki `MultiHeadPipeline` i `MultiTargetPipeline` z neutralizacją cech,
- strategie ensemble (`single`, `weighted`, `stacking`),
- diagnostykę eksperymentów w W&B (wykresy SHAP, ekspozycja cech, zbieżność HPO),
- panel EDA (Streamlit, 8 modułów analitycznych, interfejs PL/EN),
- standaryzację uruchomień przez zestaw skryptów CLI i targety `Makefile`.

Ten zakres stanowi podstawę układu kolejnych rozdziałów: najpierw architektura, następnie implementacja i konfiguracja eksperymentów, a dalej ewaluacja wyników.

2.8 Mapowanie wyzwań domenowych na decyzje projektowe

Tabela 2.2: Wyzwanie finansowe → decyzja w AlphaPulse

Wyzwanie	Ryzyko	Decyzja projektowa
Struktura era	błędny split	walidacja era-aware, purge/embargo
Niestacjonarność	niestabilny sygnał	metryki per-era + Sharpe
Duża przestrzeń konfiguracji	koszt manualny	HPO + AutoResearch
Brak standaryzacji workflow	brak reprodukowalności	YAML + CLI + artefakty
Wymóg submisji	błąd formatu	dedykowany eksport <code>predict.pkl</code>

2.9 Wnioski

Przedstawiony kontekst pokazuje, że skuteczne modelowanie w Numerai wymaga połączenia trzech perspektyw: poprawności metodologicznej, automatyzacji eksperymentów oraz inżynierii oprogramowania zapewniającej reprodukowalność. Z tego powodu w dalszej części pracy *AlphaPulse* jest traktowany nie jako pojedynczy model, lecz jako system badawczo-produkcyjny wspierający iteracyjne budowanie i ocenę pipeline'ów ML.

Rozdział 3

Architektura systemu AlphaPulse

Celem rozdziału jest przedstawienie architektury frameworku *AlphaPulse* jako konfigurowalnego systemu AutoML dla danych finansowych Numerai. Opis obejmuje podział na warstwy, przepływ danych, główne komponenty wykonawcze oraz sposób, w jaki architektura wspiera reprodukowalność i iteracyjny rozwój eksperymentów.

3.1 Założenia architektoniczne

Projekt *AlphaPulse* został oparty na kilku kluczowych założeniach:

- **Konfigurowalność:** eksperymenty są definiowane deklaratywnie, bez każdorazowej modyfikacji kodu źródłowego.
- **Modularność:** preprocessing, modele i strategie ensemble są odseparowane i mogą być dowolnie komponowane.
- **Reprodukowalność:** uruchomienia mają deterministyczny charakter przy ustalonej konfiguracji i ziarnie losowym.
- **Rozszerzalność:** nowe komponenty mogą być dodawane bez naruszania istniejącego przepływu.
- **Automatyzacja:** architektura wspiera zarówno klasyczne eksperymenty, jak i zautomatyzowane przeszukiwanie konfiguracji (HPO, AutoResearch).

3.2 Widok warstwowy systemu

Architekturę systemu można opisać jako układ pięciu współpracujących warstw.

3.2.1 Warstwa danych

Warstwa danych odpowiada za:

- pobranie i lokalne utrzymanie zbiorów Numerai,

- ładowanie podziałów (np. train/validation),
- przygotowanie macierzy cech, celu i metadanych (era),
- kontrolę parametrów wejściowych (np. subsampling, feature set).

Ta warstwa dostarcza ustandaryzowany interfejs wejściowy dla całego pipeline'u.

3.2.2 Warstwa konfiguracji eksperymentu

Konfiguracja eksperymentu jest deklarowana w formacie YAML (schema-driven). Opisuje m.in.:

- źródło danych i parametry podziału,
- zestawy cech i grupy cech,
- listę preprocessingu,
- typy modeli oraz ich hiperparametry,
- strategię ensemble,
- parametry treningu i ewaluacji.

Dzięki temu logika eksperymentu jest jawna i łatwa do wersjonowania.

3.2.3 Warstwa pipeline'u modelowego

Warstwa pipeline'u realizuje sekwencję: `Data` $\rightarrow$ `Preprocessing` $\rightarrow$ `Model(e)` $\rightarrow$ `Ensemble` $\rightarrow$ `Prediction`. W ramach tej warstwy:

- preprocessory przekształcają cechy wejściowe,
- modele bazowe uczą się predykcji celu,
- ensemble agreguje sygnały modeli cząstkowych.

Pipeline jest budowany dynamicznie na podstawie konfiguracji, co umożliwia szybkie porównywanie wielu wariantów architektury modelowej.

3.2.4 Warstwa optymalizacji i automatyzacji

System wspiera dwa poziomy automatyzacji:

- **HPO**: przeszukiwanie przestrzeni hiperparametrów i konfiguracji komponentów z użyciem Optuna TPE (flaga `--local`) lub Ray Tune (tryb domyślny rozproszony). Każda próba działa w izolowanym podprocesie; stan prób jest persystowany w SQLite (`TrialDB`), co umożliwia wznowienie przerwanej serii (`--resume`). Głównym celem optymalizacji w badaniu jest oficjalny `numera1_corr_sharpe`; raport obejmuje także średni CORR, maksymalne obsunięcie, udział dodatnich er oraz MMC na niezależnym przedziale.
- **AutoResearch**: agentowa pętla badawcza, która proponuje mutacje konfiguracji (dodanie lub usunięcie modelu, zmianę preprocessingu, ensemble albo neutralizację) i ocenia ich wpływ na metryki.

Wynikiem działania tej warstwy jest ranking prób, najlepsza konfiguracja (`best_config.json`), historia decyzji eksperymentalnych oraz diagnostyka w Weights & Biases.

3.2.5 Warstwa ewaluacji i diagnostyki

Warstwa ewaluacji obejmuje:

- **Backtester** i **EraSplitEvaluator** z opcjonalną neutralizacją cech (`FeatureNeutralizer`) przed obliczeniem metryk,
- walidację krzyżową *PurgedEraCV* oraz walk-forward z *purge/embargo*,
- raporty SHAP (`shap_report.py`) i ważności cech per-era,
- logowanie wykresów diagnostycznych do W&B (`wandb_diagnostics.py`): korelacja per-era, ekspozycja cech, heatmapa korelacji ensemble, zbieżność HPO.

3.2.6 Warstwa artefaktów i eksportu

Warstwa artefaktów odpowiada za:

- zapisy metryk i logów eksperymentów,
- serializację najlepszych konfiguracji,
- eksport predykcji do formatu wymaganej submisji (np. `predict.pkl`).

To domyka przepływ od eksperymentu badawczego do artefaktu konkursowego.

3.3 Przepływ wykonania end-to-end

Typowy scenariusz działania systemu obejmuje następujące etapy:

1. pobranie danych i przygotowanie lokalnego katalogu roboczego,
2. uruchomienie eksperymentu z pliku YAML,
3. trenowanie i ewaluacja modeli bazowych oraz ensemble,
4. (opcjonalnie) uruchomienie HPO lub AutoResearch,
5. wybór najlepszej konfiguracji i eksport predykcji.

W ujęciu operacyjnym odpowiada temu zestaw dedykowanych skryptów CLI, które oddzielają definicję eksperymentu od mechaniki wykonania.

3.4 Komponenty funkcjonalne i ich odpowiedzialności

Architektura dzieli odpowiedzialności pomiędzy komponenty:

- **Data Loader**: odczyt danych i przygotowanie splitów,
- **Experiment Schema**: walidacja konfiguracji wejściowej,
- **Pipeline Builder**: konstrukcja grafu preprocessingu, modeli i ensemble,
- **Trainer/Evaluator**: trening i obliczanie metryk jakości,
- **HPO Engine**: zarządzanie próbami optymalizacyjnymi,
- **AutoResearch Loop**: agentowe sterowanie kolejnymi eksperymentami,
- **Exporter**: generowanie artefaktów konkursowych.

Takie rozdzielenie upraszcza testowanie i ułatwia rozwój systemu.

3.5 Wzorce zapewniające jakość inżynierską

W *AlphaPulse* zastosowano praktyki zwiększające niezawodność:

- **Separation of concerns** między konfiguracją, logiką uczenia i eksportem,
- **CLI-first workflow** dla łatwej automatyzacji uruchomień,

- **Jednoznaczne artefakty wyjściowe** dla porównywania eksperymentów,
- **Integracja testów i narzędzi jakości kodu** dla stabilności rozwoju.

W efekcie architektura nadaje się zarówno do badań iteracyjnych, jak i do utrzymania dłuższej linii rozwojowej projektu.

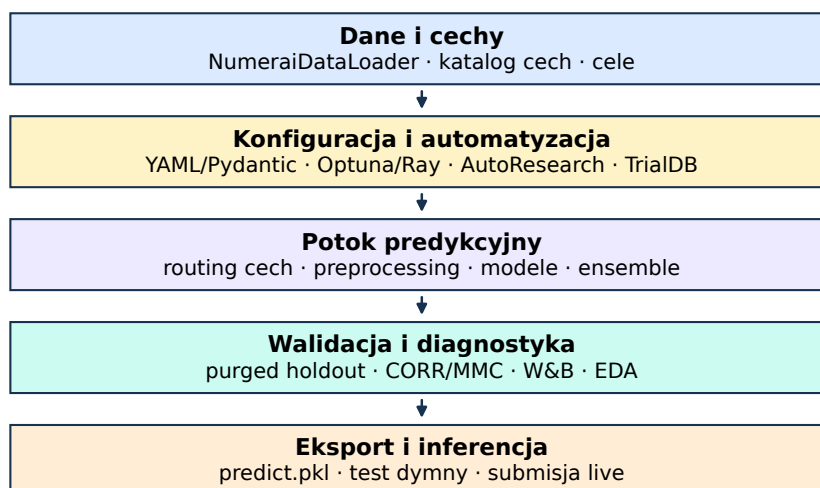
3.6 Ograniczenia architektury

Pomimo wysokiej elastyczności architektura ma ograniczenia:

- skuteczność zależy od jakości zdefiniowanej przestrzeni konfiguracji,
- duża liczba wariantów zwiększa koszt obliczeniowy eksperymentów,
- automatyzacja agentowa wymaga kontroli jakości proponowanych mutacji.

Ograniczenia te są naturalne dla systemów AutoML i stanowią punkt wyjścia do dalszej optymalizacji.

3.7 Diagram architektury logicznej



Rysunek 3.1: Uproszczona architektura logiczna AlphaPulse. Diagram wygenerowano z aktualnej struktury implementacji; ograniczenie liczby komponentów zapewnia czytelność w druku.

Oprócz diagramu warstwowego, uproszczony przepływ end-to-end można przedstawić jako: Dane → YAML → Pipeline → HPO / AutoResearch → Artefakty / Export.

3.8 Kontrakty komponentów

Kontrakty komponentów określają minimalne wymagania wejścia i wyjścia dla kolejnych warstw systemu. Dzięki temu pipeline może być rozwijany modularnie: nowy loader, preprocessor, model lub strategia ensemble musi spełnić ten sam interfejs logiczny, bez konieczności zmiany pozostałych części frameworku.

Tabela 3.1: Podstawowe kontrakty komponentów systemu AlphaPulse

Komponent	Wejście	Wyjście
Loader danych	konfiguracja zbioru Numerai, wersja danych, lista cech	macierze X , y , kolumna era oraz metadane zbioru
Preprocessor	dane treningowe i parametry transformacji	przekształcone cechy oraz zapisany stan transformacji
Model bazowy	cechy, target, konfiguracja hiperparametrów	wytrenowany estymator i predykcje walidacyjne
Ensemble	predykcje modeli bazowych i konfiguracja łączenia	finalna predykcja oraz metadane wag lub meta-modelu
Eksport	predykcje końcowe i identyfikatory przykładów	plik predict.pkl zgodny z wymaganiami Numerai

Konfiguracja eksperymentu pełni rolę wspólnego kontraktu sterującego. Wymagane sekcje YAML obejmują **data**, **features**, **preprocessing**, **models**, **ensemble_method**, **train** oraz **evaluation**. Każde uruchomienie zapisuje konfigurację, metryki, logi prób oraz, zależnie od trybu pracy, artefakty takie jak **best_config.json** i **predict.pkl**.

Architektura *AlphaPulse* realizuje podejście *configuration-driven* i integruje pełny cykl pracy konkursowej: od danych, przez eksperymenty i automatyzację, po eksport predykcji. Kluczową wartością systemu jest połączenie modularności technicznej z reprodukowalnością badawczą, co stanowi fundament dalszych rozdziałów implementacyjnych i eksperymentalnych.

Rozdział 4

Implementacja systemu AlphaPulse

Celem rozdziału jest szczegółowe przedstawienie implementacji frameworku *AlphaPulse*: sposobu organizacji kodu, mechanizmów konfiguracji eksperymentów, interfejsów uruchomieniowych oraz realizacji kluczowych etapów pipeline’u od danych do artefaktu submisyjnego.

4.1 Organizacja repozytorium

Implementacja projektu została podzielona na warstwy odpowiadające cyklowi pracy eksperymentalnej:

- `src/alphapulse/` — kod domenowy (pipeline, HPO, ewaluacja, `AutoResearch`, `features/`),
- `scripts/` — skrypty wykonawcze CLI dla najważniejszych scenariuszy,
- `experiments/` — deklaratywne definicje eksperymentów (YAML),
- `eda/` — samodzielny panel Streamlit do analizy eksploracyjnej,
- `tests/` — testy jednostkowe i integracyjne (w tym metryki MMC),
- `artifacts/` — wyniki uruchomień, konfiguracje i eksporty.

Taki podział rozdziela logikę frameworku od warstwy operacyjnej (uruchamianie), co zwiększa czytelność i ułatwia utrzymanie projektu.

4.2 Warstwa uruchomieniowa (CLI)

4.2.1 Skrypty wykonawcze

System udostępnia zestaw skryptów CLI obsługujących pełny przepływ:

- `download_dataset.py` — pobieranie danych Numerai,

- `run_experiment.py` — uruchamianie eksperymentów YAML (z opcją `--wandb-project`),
- `hpo_pipeline.py` — HPO (Optuna lokalnie lub Ray Tune),
- `autoresearch.py` — pętla agentowa AutoResearch,
- `run_test_pipeline.py` — lekki test end-to-end,
- `export_numerai_pickle.py`, `export_from_yaml.py` — eksport `predict.pkl`,
- `live_inference.py` — predykcje na `live.parquet`,
- `submit_predictions.py` — upload predykcji do API Numerai,
- `make_feature_groups.py` — generowanie grup cech z `features.json`.

Skrypty pełnią rolę cienkiej warstwy orkiestracji: pobierają argumenty, walidują wejście, inicjują komponenty frameworku i zapisują artefakty.

4.2.2 Konwencja parametrów wejściowych

Interfejs CLI wykorzystuje jednolitą konwencję argumentów:

- ścieżki danych i artefaktów (`--data-dir`, `--output-dir`),
- parametry skali eksperymentu (`--train-subsample`, liczba prób),
- wskazanie konfiguracji źródłowej (`--config`, `--best-config-path`),
- tryb lokalny lub rozproszony (zależnie od skryptu).

Ujednolicenie argumentów upraszcza automatyzację i integrację z narzędziami CI/CD.

4.3 Konfiguracja eksperymentów (YAML)

4.3.1 Schemat i walidacja

Eksperymenty definiowane są deklaratywnie. Schemat obejmuje sekcje:

- `data`,
- `features`,
- `preprocessing`,

- `models`,
- `ensemble_method/ensemble_params`,
- `train`,
- `evaluation`.

Każde uruchomienie przechodzi walidację zgodności konfiguracji, co ogranicza liczbę błędów wykonania i gwarantuje spójność eksperymentów.

4.3.2 Konfigurowalność bez zmian kodu

Najważniejszą własnością implementacji jest możliwość modyfikacji eksperymentu poprzez edycję YAML:

- zmiana modelu i hiperparametrów,
- podmiana preprocessingu,
- przełączenie strategii ensemble,
- zmiana metryki głównej ewaluacji.

Pozwala to prowadzić szybkie iteracje badawcze przy zachowaniu pełnej ścieżki audytu zmian w repozytorium.

4.4 Implementacja warstwy danych

4.4.1 Loader danych Numerai

Warstwa danych dostarcza zunifikowany interfejs ładowania splitów oraz metadanych. Implementacja obejmuje:

- odczyt plików parquet i plików opisowych,
- wybór zestawu cech i subsamplingu,
- przygotowanie struktur `X`, `y`, `era`.

Taki kontrakt danych jest wykorzystywany przez wszystkie wyższe warstwy pipeline’u.

4.4.2 Obsługa danych wejściowych pod eksperymenty

Loader obsługuje zarówno pełne zbiory, jak i tryby szybkich testów na podzbiorach danych. To umożliwia:

- szybkie iteracje deweloperskie,
- tańsze testy poprawności potoku,
- stopniowe skalowanie kosztownych eksperymentów.

4.5 Implementacja pipeline’u modelowego

4.5.1 Preprocessing

Implementacja preprocessingu wykorzystuje listę kroków konfigurowanych w YAML. Wspierane są m.in.:

- skalowanie (*StandardScaler*, *RobustScaler*),
- redukcja wymiarowości (*PCA*, *TruncatedSVD*),
- selekcja cech i transformacje regularizujące.

Kroki preprocessingu są wykonywane deterministycznie i mogą być łączone w sekwencje.

4.5.2 Modele bazowe

Warstwa modeli została zaprojektowana jako zestaw wymiennych adapterów (spójny interfejs *fit/predict*) dla różnych rodzin:

- gradient boosting,
- modele drzewiaste,
- modele liniowe,
- wybrane modele fundamentowe dla danych tabelarycznych.

Standaryzacja interfejsu upraszcza porównywanie modeli i ich integrację z ensemble.

4.5.3 Strategie ensemble

Implementacja obsługuje trzy podstawowe tryby:

- `single` – pojedynczy model,
- `weighted` – ważona agregacja predykcji,
- `stacking` – meta-model uczony na predykcjach bazowych [14].

Strategia ensemble jest traktowana jako pełnoprawny element konfiguracji eksperymentu.

4.6 Implementacja HPO

4.6.1 Silniki optymalizacji

HPO realizowane jest przez moduł `src/alphapulse/hpo/` z dwoma trybami uruchomienia:

- **Tryb lokalny** (`--local`): Optuna TPE [8] (Bayesowski sampler) z izolacją każdej próby w osobnym podprocesie; baza `optuna.db` i `trials.db` (TrialDB).
- **Tryb rozproszony**: Ray Tune [9] z callbackiem W&B; wymaga zależności `alphapulse[hpo]`.

Wspólne elementy obu trybów:

- definicja przestrzeni przeszukiwania (`search_space.py`, `registry.py`),
- funkcja celu (`objective.py`) z oceną era-holdout lub walk-forward (3 foldy),
- routing cech (`feature_routing.py`) i strategię wielocelowe (`target_strategy.py`),
- limit czasu próby (`--trial-timeout`, domyślnie 3600 s) i budżet godzinowy (`--max-hours`),
- eksport najlepszej konfiguracji (`hpo/export.py`).

4.6.2 Śledzenie wyników prób

Każda próba HPO generuje komplet informacji eksperymentalnych:

- parametry wejściowe i hash konfiguracji,

- oficjalne diagnostyki `numera_i_corr_*` i `numera_i_mmc_*` dla holdoutu oraz niezależnego przedziału walidacyjnego,
- wykresy diagnostyczne w W&B (ważność cech, ekspozycja, korelacja per-era),
- metadane wykonania (czas, typ modelu, ewentualny błąd).

Po zakończeniu sweepu logowany jest run `best-trial-diagnostics` oraz wykresy `search-convergence` i `hpo-summary` (scatter: corr Sharpe vs. MMC Sharpe vs. czas).

4.7 Implementacja AutoResearch

4.7.1 Pętla agentowa

AutoResearch rozszerza klasyczny HPO o mechanizm agentowy:

1. analiza dotychczasowych wyników,
2. propozycja mutacji konfiguracji,
3. uruchomienie kolejnej próby,
4. aktualizacja stanu badania.

Stan eksperymentu i decyzje agenta są zapisywane w artefaktach, co pozwala audytować przebieg procesu badawczego.

4.7.2 Rola AutoResearch w implementacji

Mechanizm ten nie zastępuje pełni pracy badacza, lecz automatyzuje najbardziej powtarzalny etap: generowanie kolejnych hipotez konfiguracyjnych i ich testowanie. Wpływ tej automatyzacji na czas pracy wymaga kontrolowanego pomiaru i nie jest rozstrzygany na podstawie samej implementacji.

4.8 Eksport predykcji i artefakty końcowe

Końcowym etapem implementacji jest produkcja artefaktu submisyjnego:

- odtworzenie najlepszego pipeline’u z `best_config.json` lub YAML,
- trening na wskazanym zbiorze z zachowaniem kontraktu schematu cech,
- wygenerowanie predykcji z normalizacją rangową per-era,

- serializacja do `predict.pkl` z nazwą kanoniczną `<TIMESTAMP>_<ARCH>_<TARGET>_<CONFIG_HASH>_predict.pkl` i symlinkiem `latest_predict.pkl`,
- zapis `*_provenance.json`: rozwiązana konfiguracja, snapshot zależności (`uvexport`) i hash commita Git,
- test dymny: czysty podproces ładuje pickle i wykonuje forward pass na syntetycznej ramce danych.

4.8.1 Workflow produkcyjny (inferencja i submisja)

Tygodniowy cykl turniejowy realizowany jest trzema krokami:

1. pobranie świeżych danych (`live.parquet`),
2. `live_inference.py` — predykcje z walidacją formatu (`--validate`),
3. `submit_predictions.py` — upload CSV (`id, prediction`) do API Numerai.

Skrypty wymagają kluczy `NUMERAI_PUBLIC_API_KEY` i `NUMERAI_PRIVATE_API_KEY` (lokalny plik `.env` lub zmienne środowiskowe).

4.9 Zapewnienie jakości kodu

W implementacji zastosowano standardowe mechanizmy jakości:

- linting i formatowanie (`ruff, make fmt, make lint`),
- statyczne sprawdzanie typów (`mypy, make types`),
- testy automatyczne z pokryciem (`pytest, make test`),
- wykrywanie martwego kodu (`vulture, make deadcode`),
- hooki pre-commit i target `make check` (`lint + types + test + deadcode`),
- osobna ścieżka jakości dla modułu EDA (`make eda-lint`).

To ogranicza regresje i zwiększa stabilność frameworku w kolejnych iteracjach.

4.10 Przykładowa konfiguracja referencyjna

```
version: "1"
data:
  data_dir: data/v5.3
  train_subsample: 0.10
  target_col: target_ender_60
  seed: 20260824
models:
  - type: XGBoost
    params:
      max_depth: 3
      learning_rate: 0.05
      tree_method: hist
ensemble_method: single
evaluation:
  primary_metric: numerai_corr_sharpe
```

4.11 Przestrzeń przeszukiwania HPO

Przykładowe parametry optymalizowane przez HPO (Optuna/Ray Tune):

- typ modelu (XGBoost/LightGBM/CatBoost/Ridge/...),
- hiperparametry modelu (`max_depth`, `learning_rate`, `n_estimators`),
- wybór preprocessingu i jego parametrów,
- metoda ensemble i parametry meta-modelu.

W badaniu końcowym przestrzeń ograniczono do jednego modelu z puli XGBoost, LightGBM, CatBoost, TabPFN i TabICL. Po 20 próbach inicjalnych sampler TPE korzystał z dotychczasowej historii, a każda konfiguracja była oceniana w tym samym protokole czasowym. Szczegółowy budżet opisano w rozdziale 5.

Rozdział 5

Metodyka badań

Rozdział opisuje faktycznie wykonane serie, reguły włączenia obserwacji do analizy oraz sposób wyboru kandydatów. Celem jest rozdzielenie dwóch rodzajów dowodów: weryfikacji skalowalności całego systemu i porównania jakości konfiguracji w jednolitym protokole Numerai v5.3.

5.1 Pytania eksperymentalne

Analiza odpowiada na cztery pytania:

1. Czy **AlphaPulse** wykonuje kompletny potok na pełnym zbiorze danych i zapisuje artefakty umożliwiające odtworzenie uruchomienia?
2. Które rodziny modeli osiągają najwyższy oficjalny CORR w ustalonym budżecie i podziale czasowym?
3. Jak zgodne są rankingi na purged holdoutcie i późniejszym przedziale walidacyjnym oraz które konfiguracje tworzą kompromis CORR–MMC?
4. Jaki koszt wykonania i poziom kompletności prób uzyskano w serii HPO?

Porównania rodzin mają charakter opisowy. TPE wybiera konfiguracje adaptacyjnie, dlatego liczebności grup nie są równe, a obserwowane różnice nie są traktowane jako kontrolowana ablacja ani dowód przyczynowy.

5.2 Dwie serie o rozłącznych rolach

Materiał empiryczny obejmuje benchmark pełnoskalowy i końcową serię HPO (tabela 5.1). Różne wersje danych i cele wykluczają wspólny ranking jakości. Pierwsza seria odpowiada na pytanie o wykonalność i czas pełnej ścieżki, druga — o jakość konfiguracji według oficjalnych metryk.

Benchmark wykorzystał wszystkie 574 ery uczące. Potok **StandardScaler** oraz **EraEnsemble** z dziesięcioma modelami **XGBoost** przetworzył następnie 3 895 460 wierszy walidacyjnych w 1888,6 s, czyli 31,5 min. Wynik ten dokumentuje możliwość pracy na pełnej skali; jego miary jakości nie są przenoszone do rankingu v5.3.

Tabela 5.1: Serie wykorzystane w badaniu i ich role

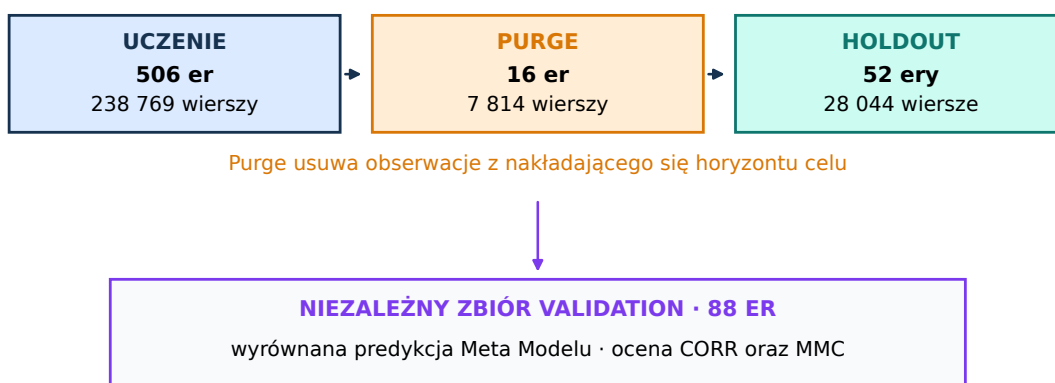
Seria	Dane	Cel	Zakres train	Rola
Benchmark pełnoskalowy	v5.2	<code>target</code>	2 746 268 wierszy, 780 cech	skalowalność i kompletność przepływu
Końcowe HPO	v5.3	<code>target_ender_60</code>	274 627 wierszy, 780 cech	porównanie oficjalnego CORR i MMC

5.3 Dane i podział czasowy badania końcowego

Końcowe HPO korzysta z deterministycznej, dziesięcioprocentowej próby zbioru `train.parquet`: 274 627 wierszy, 574 ery i 780 cech bazowych. Losowanie ma stałe ziarno 20260824, a integralność wejścia jest utrwalona skrótami SHA-256 w pliku `protocol.json`. Każda próba przewiduje `target_ender_60`.

Ostatnie 52 ery próbki tworzą holdout. Szesnaście bezpośrednio poprzedzających er zostaje usuniętych z uczenia ze względu na nakładający się horyzont celu. W efekcie część ucząca obejmuje 506 er i 238 769 wierszy, strefa separacji 7814 wierszy, a holdout 28 044 wiersze. Drugim punktem oceny jest niezależny fragment `validation.parquet`: 88 er, dla których dostępna jest wyrównana predykcja Meta Modelu. Umożliwia to obliczenie zarówno CORR, jak i MMC.

Próba ucząca v5.3: 274 627 wierszy, 574 ery, 780 cech bazowych



Rysunek 5.1: Podział czasowy badania końcowego. Strefa 16 er oddziela uczenie od holdoutu; zewnętrzny przedział 88 er służy do oceny transferu oraz MMC.

5.4 Protokół HPO

Parametry protokołu zostały zapisane przed serią i nie zmieniały się między próbami. Najważniejsze ustawienia przedstawia tabela 5.2.

Tabela 5.2: Protokół końcowej serii HPO

Element	Ustawienie
Wersja danych i cel	Numerai v5.3, <code>target_ender_60</code>
Próba i ziarno	10%, 274 627 wierszy, seed 20260824
Podział	506 er train, 16 er purge, 52 ery holdout
Walidacja zewnętrzna	88 er z predykcją Meta Modelu
Funkcja celu	oficjalny <code>numerai_corr_sharpe</code> na holdoucie
Sampler	Optuna TPE, 20 prób inicjalnych
Przestrzeń modeli	XGBoost, LightGBM, CatBoost, TabPFN, TabICL
Złożoność próby	maksymalnie jeden model, routing grup cech
Limity	200 prób lub 11,5 h; 1800 s na próbę
Implementacja metryk	<code>numerai-tools==0.6.0</code>
Wersja kodu	commit 1a86893, protokół <code>corrected-hpo-2026-08-v5</code>

Do analizy włączono wyłącznie rekordy ze statusem `completed`, kompletną konfiguracją i skończonym wektorem trzech kluczowych miar: CORR Sharpe na holdoucie, CORR Sharpe na validation oraz MMC Sharpe na validation. Wcześniej-sze uruchomienia kontrolne z identycznym ziarnem i konfiguracją nie są liczone ponownie.

5.5 Przestrzeń konfiguracji

Jedna próba wybiera rodzinę modelu, grupy cech, skalowanie, opcjonalną selekcję wariancji, parametry regularizacji, liczbę iteracji i stopień neutralizacji względem cech. Routing łączy zestawy bazowe (`small`, `medium`) z grupami tematycznymi z katalogu Numerai. Liczba wejściowych cech różni się zatem między konfiguracjami.

Dla modeli boostingowych optymalizowane są między innymi: tempo uczenia, głębokość lub liczba liści, minimalny rozmiar liścia, subsampling kolumn i wierszy oraz regularyzacja. TabPFN i TabICL działają w profilach pamięciowych ograniczających liczbę przykładów oraz wymiar po redukcji. Ich wyniki opisują zachowanie w tym konkretnym budżecie, a nie ogólną przewagę jednej architektury nad drugą.

5.6 Metryki i reguła selekcji

Główną metryką jest oficjalny CORR Sharpe z równania (2.1) na 52-erowym holdoutcie. Dla obu przedziałów raportowane są średni CORR, odchylenie, udział dodatnich er i maksymalne obsunięcie. Na 88-erowym validation obliczany jest dodatkowo oficjalny MMC Sharpe.

Nie tworzą syntetycznej funkcji łączącej CORR i MMC. Zamiast tego wyróżniono:

1. lidera CORR Sharpe na holdoutcie,
2. lidera CORR Sharpe na niezależnym validation,
3. lidera MMC Sharpe na validation,
4. zbiór Pareto względem CORR i MMC na validation.

Taka reguła zachowuje interpretowalność kompromisu między siłą sygnału i jego wkładem niezależnym od Meta Modelu.

5.7 Metody analizy

Dla każdej kompletnej próby zestawiono metryki, czas wykonania, rodzinę modelu i liczbę trasowanych cech. Zgodność dwóch przedziałów czasowych oceniono korelacją Pearsona oraz korelacją rang Spearmana. Rodziny modeli porównano za pomocą median, wykresów pudełkowych i wszystkich punktów, zawsze z podaniem liczebności.

Front Pareto wyznaczono przez dominację dwukryterialną: konfiguracja należy do frontu, jeżeli nie istnieje inna próba o co najmniej takim samym CORR Sharpe i MMC Sharpe oraz wyniku ściśle lepszym w co najmniej jednym kryterium. Przebieg HPO analizowano przez maksimum narastające, rozdzielając 20 prób inicjalnych i fazę adaptacji TPE.

5.8 Reprodukowalność

Kompletna proveniencja końcowej serii znajduje się w plikach `protocol.json`, `trials.db` i `optuna.db`. Protokół zawiera wersję Pythona, commit, skróty kodu i danych, seed, liczbę próbkowanych wierszy, cel oraz parametry oceny. Konfiguracja każdej próby i jej metryki są przechowywane w pojedynczym rekordzie SQLite, a przebieg jest równolegle rejestrowany w W&B.

Minimalne odtworzenie badania w środowisku projektu wymaga następujących poleceń:

```
uv sync --extra dev --extra hpo --extra foundation
uv run python scripts/download_dataset.py --config.dataset-version v5.3
    --config.output-dir data
uv run python scripts/hpo_pipeline.py --data-dir data/v5.3 --target-col
    target_ender_60 --train-subsample 0.10 --seed 20260824 --num-trials 200
    --output-dir artifacts/hpo_v53_final --local --objective numerai_corr_sharpe
    --purge-eras 16 --max-hours 11.5 --trial-timeout 1800 --sampler tpe
    --n-startup-trials 20 --max-models 1 --gpu --wandb-project
    alphapulse-v53-final --no-wandb-diagnostics
```

5.9 Zagrożenia dla trafności

Badanie końcowe obejmuje jedno ziarno i 27 kompletnych konfiguracji. Adaptacyjny sampler generuje nierówne liczebności rodzin, więc agregaty nie stanowią testu istotności ani kontrolowanej ablacji. HPO korzysta z 10% danych uczących; wyniki nie są ekstrapolowane numerycznie na pełny trening. Benchmark v5.2 dowodzi skalowalności, lecz z powodu innego celu i wersji danych nie stanowi porównania jakościowego.

Holdout był wielokrotnie wykorzystywany przez funkcję celu, dlatego rozstrzygnięcie praktyczne opiera się również na późniejszym przedziale validation. Jego 88 er zwiększa niezależność oceny, ale nadal reprezentuje jeden zakres rynkowy. Wdrożenie produkcyjne powinno poprzedzać ponowne uczenie wybranej receptury na pełnych danych oraz monitoring wielu kolejnych rund live.

Rozdział 6

Wyniki eksperymentów

Rozdział przedstawia wynik benchmarku pełnoskalowego oraz analizę końcowej serii HPO. Wszystkie rankingi jakości pochodzą z jednego protokołu v5.3 i oficjalnych metryk Numerai. Starsze uruchomienia kontrolne nie są łączone z tą populacją.

6.1 Weryfikacja pełnej skali

Benchmark pełnoskalowy objął 2 746 268 wierszy uczących, 780 cech i 574 ery. Potok z dziesięcioma partycjami XGBoost wykonał trening, predykcję dla 3 895 460 wierszy walidacyjnych, ocenę i zapis diagnostyki w 1888,6 s (31,5 min). Utworzone zostały konfiguracja rozwiązana, hash konfiguracji i pełny zapis przebiegu. Rezultat potwierdza, że ta sama architektura obsługuje nie tylko próby HPO, lecz także trening na całym dostępnym zbiorze.

Tabela 6.1: Wynik operacyjny benchmarku pełnoskalowego

Wielkość	Wartość
Wiersze train / validation	2 746 268 / 3 895 460
Liczba cech / er train	780 / 574
Modele składowe	10 × XGBoost
Czas całkowity	1888,6 s (31,5 min)

6.2 Kompletność końcowej serii HPO

Seria została zamknięta po zapisaniu 32 prób. Kryteria kompletności spełniło 27 konfiguracji (84,4% całej puli); cztery rekordy nie zawierały wyniku, a jeden pozostał niedokończony. Analiza ilościowa obejmuje wyłącznie 27 kompletnych rekordów.

Łączny czas ukończonych prób wyniósł 1,44 h, a mediana czasu pojedynczej konfiguracji 127,2 s. Są to czasy prób modelowych, bez jednorazowego pobrania danych i przygotowania środowiska.

Tabela 6.2: Kompletność końcowej serii HPO

Status	Liczba	Udział
Kompletna próba	27	84,4%
Bez kompletnego wyniku	4	12,5%
Niedokończona	1	3,1%
Razem	32	100,0%

6.3 Rozkład oficjalnych metryk

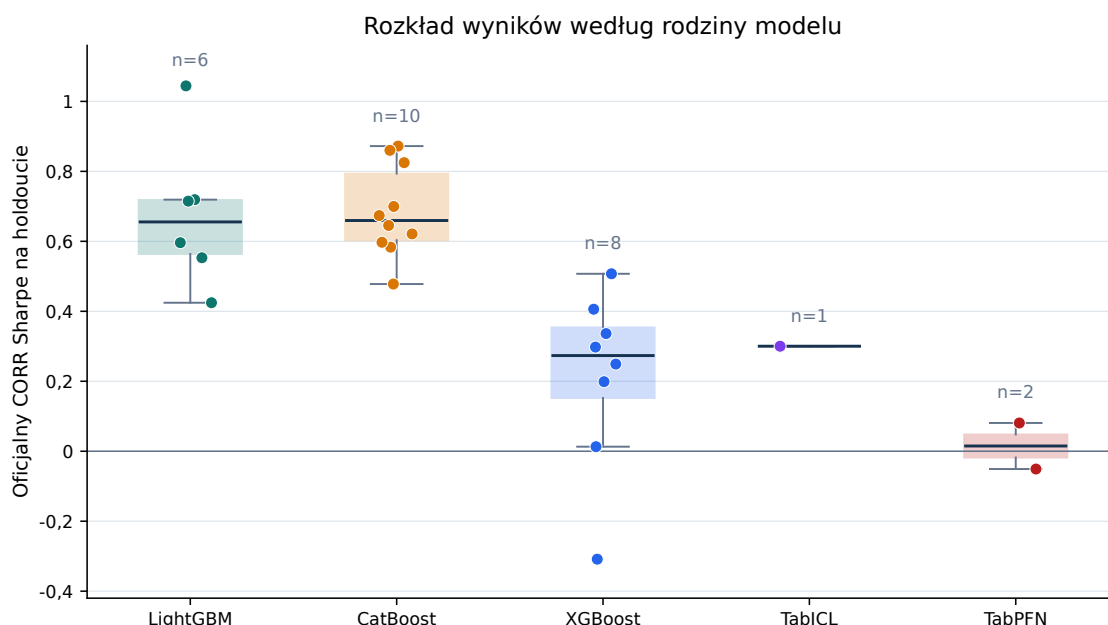
Tabela 6.3 syntetyzuje wszystkie kompletne konfiguracje. Na holdoucie dodatni CORR Sharpe uzyskało 25 z 27 prób. Na późniejszym przedziale validation dodatni CORR Sharpe osiągnęły 24 próby, natomiast dodatni MMC Sharpe 13 prób. Różnica pokazuje, że sama korelacja z celem nie gwarantuje unikalności sygnału względem Meta Modelu.

Tabela 6.3: Podsumowanie oficjalnych metryk dla 27 kompletnych prób

Metryka	Średnia	Mediana	Wynik dodatni
CORR Sharpe, holdout (52 ery)	0,479	0,553	25/27
CORR Sharpe, validation (88 er)	0,128	0,128	24/27
MMC Sharpe, validation (88 er)	0,0247	-0,0098	13/27

6.4 Rodziny modeli w ustalonym budżecie

Najwyższe mediany CORR Sharpe na holdoucie uzyskały CatBoost (0,659; $n = 10$) i LightGBM (0,655; $n = 6$). Mediana XGBoost wyniosła 0,273 ($n = 8$), TabICL 0,300 ($n = 1$), a TabPFN 0,015 ($n = 2$). Wszystkie obserwacje i rozrzut przedstawia rysunek 6.1.

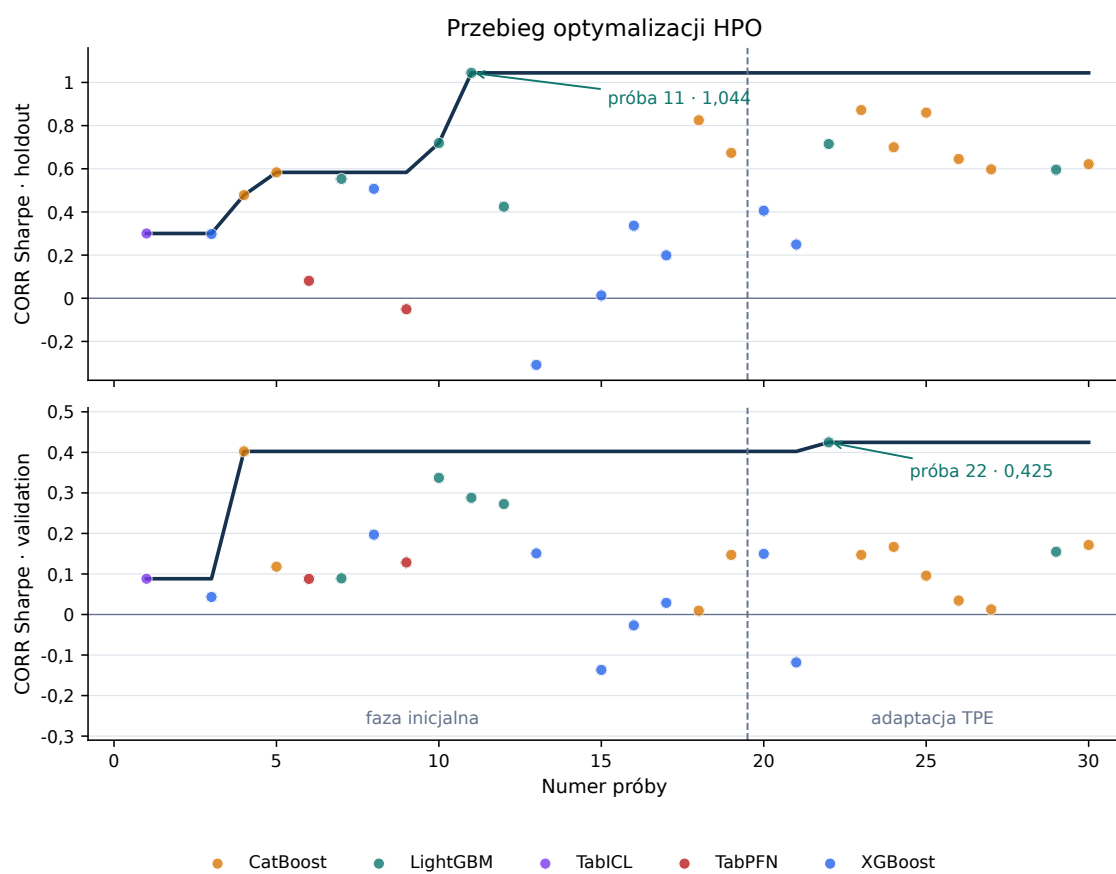


Rysunek 6.1: Oficjalny CORR Sharpe na holdoucie według rodziny modelu. Punkty oznaczają kompletne próby, a liczności podano nad grupami.

Na validation mediana LightGBM wyniosła 0,280, CatBoost 0,133, XGBoost 0,036, TabPFN 0,108, a TabICL 0,088. Wynik jest opisem badanego budżetu, nie rankingiem architektur w ogólności: TPE przydzielił rodzinom różne liczby prób, a modele fundamentowe korzystały z limitów liczby przykładów i redukcji wymiaru.

6.5 Przebieg optymalizacji

Pierwsze 20 konfiguracji stanowiło fazę inicjalną TPE. Lider holdoutu pojawił się już w próbie 11. Po przejściu do fazy adaptacyjnej poprawił się natomiast najlepszy wynik na niezależnym validation: próba 22 podniosła maksimum CORR Sharpe z 0,402 do 0,425. W kolejnych próbach maksimum nie uległo zmianie (rysunek 6.2).

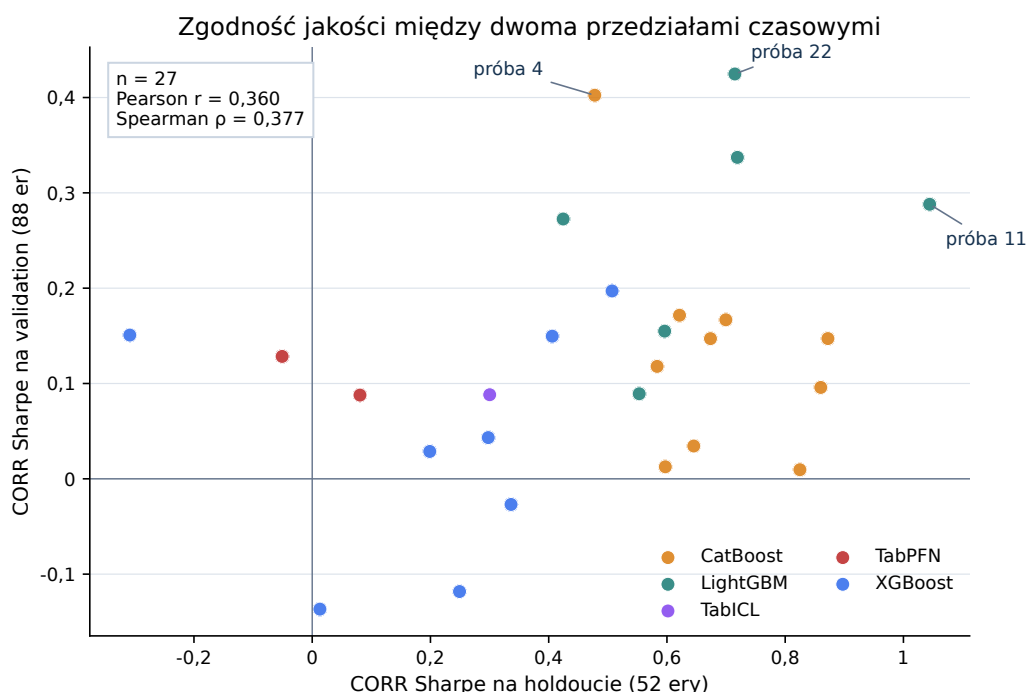


Rysunek 6.2: Wyniki prób i maksimum narastające dla holdoutu oraz validation. Linia przerywana oddziela fazę inicjalną od adaptacji TPE.

Wynik wskazuje, że adaptacja była użyteczna dla kryterium transferu, mimo że sampler optymalizował holdout. Jednocześnie brak dalszej poprawy po próbie 22 uzasadnia traktowanie odnalezionych konfiguracji jako stabilnego krótkiego rankingu kandydackiego w ramach dostępnego budżetu.

6.6 Transfer między przedziałami czasowymi

Korelacja Pearsona między CORR Sharpe na holdoucie i validation wyniosła $r = 0,360$, a korelacja rang Spearmana $\rho = 0,377$. Związek jest dodatni, lecz umiarkowany: wysoka pozycja na 52-erowym holdoucie zwiększa szansę dobrego wyniku później, ale nie zachowuje rankingu konfiguracji (rysunek 6.3).



Rysunek 6.3: Zgodność CORR Sharpe między holdoutem i niezależnym validation dla 27 kompletnych konfiguracji.

Próba 11 maksymalizuje holdout, lecz na validation ustępuje próbom 22, 4 i 10. To najważniejszy argument za dwustopniową selekcją: holdout steruje HPO, natomiast późniejszy przedział rozstrzyga o przenoszeniu sygnału w czasie.

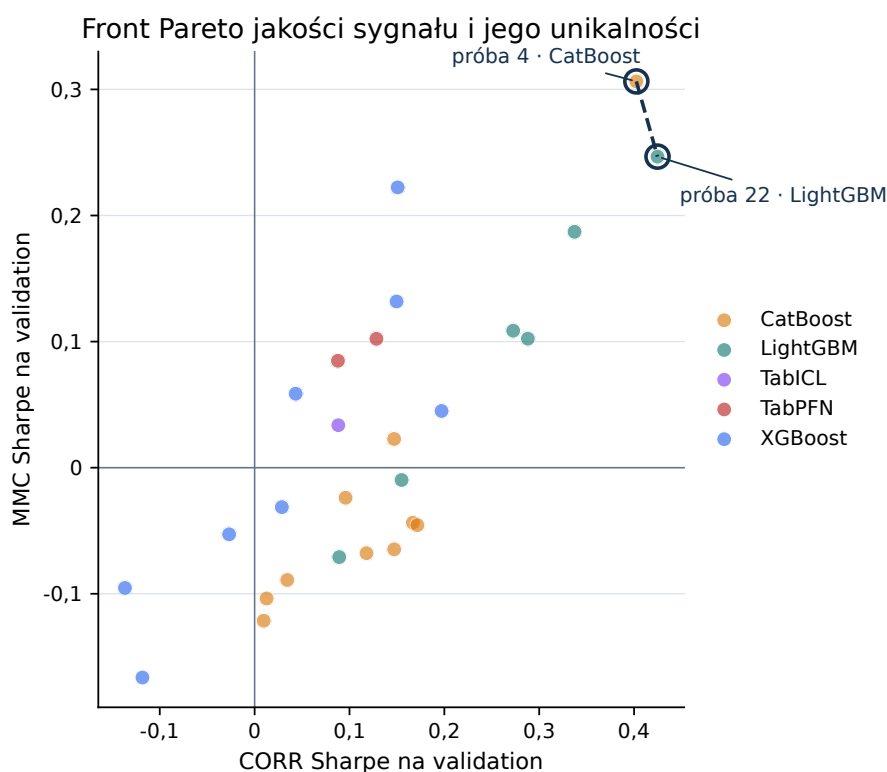
6.7 Kandydaci końcowi

Tabela 6.4 zestawia pięć reprezentatywnych konfiguracji. Próba 11 jest liderem holdoutu, próba 22 liderem CORR na validation, a próba 4 liderem MMC. Próby 10 i 8 pokazują mocne alternatywy z rodzin LightGBM i XGBoost.

Tabela 6.4: Oficjalne metryki końcowych kandydatów; HO — holdout, VAL — validation

Próba	Model	HO CORR		VAL CORR		VAL MMC	
		średnia	Sharpe	średnia	Sharpe	średnia	Sharpe
22	LightGBM	0,03032	0,7149	0,01679	0,4247	0,00802	0,2467
4	CatBoost	0,02161	0,4779	0,01483	0,4024	0,01031	0,3063
10	LightGBM	0,02743	0,7191	0,01450	0,3372	0,00716	0,1871
11	LightGBM	0,03742	1,0443	0,01038	0,2880	0,00310	0,1023
8	XGBoost	0,02047	0,5072	0,00705	0,1971	0,00142	0,0450

W przestrzeni validation CORR–MMC tylko próby 22 i 4 nie są zdominowane (rysunek 6.4). Próba 22 maksymalizuje jakość sygnału, a próba 4 jego unikalność. Nie istnieje więc jeden zwycięzca niezależny od celu wdrożeniowego.



Rysunek 6.4: Front Pareto na niezależnym validation. Okręgi wyróżniają próby niezdominowane względem CORR Sharpe i MMC Sharpe.

Do treningu nastawionego na stabilny CORR wybrano próbę 22. Jest to LightGBM z RobustScalerem, selekcją wariancji, 989 cechami z grup `medium+agility+midnight`, 200 rundami, tempem uczenia 0,0200, 31 liśćmi i neutralizacją cech 0,2373. Jej CORR Sharpe wynosi 0,7149 na holdoucie i 0,4247 na validation, przy dodatnim MMC Sharpe 0,2467.

Próba 4 jest alternatywą nastawioną na MMC. Wykorzystuje CatBoost, StandardScaler, 970 cech z sześciu grup, głębokość 5, tempo uczenia 0,0314 i neutralizację 0,4947. Osiąga CORR Sharpe 0,4024 oraz MMC Sharpe 0,3063 na validation. Próba 11 pozostaje górną granicą jakości na holdoutcie: CORR Sharpe 1,044, średni CORR 0,0374, maksymalne obsunięcie 0,0396 i 84,6% dodatnich er.

6.8 Synteza wyników

Badanie dostarczyło trzech rozstrzygnięć. Po pierwsze, pełny potok działa na milionach wierszy i zapisuje odtwarzalne artefakty w czasie poniżej 32 minut dla badanego benchmarku. Po drugie, w ograniczonym budżecie najwyższe mediany holdoutu osiągnęły LightGBM i CatBoost. Po trzecie, niezależny przedział zmienił ranking i wyłonił dwa praktyczne profile: próbę 22 dla CORR oraz próbę 4 dla MMC. Są to bezpośrednie pomiary, bez numerycznej ekstrapolacji na pełny trening.

Rozdział 7

Dyskusja i zakończenie

7.1 Realizacja celu pracy

Główny cel pracy — stworzenie konfigurowalnego środowiska AutoML dla finansowych potoków Numerai — został zrealizowany na poziomie architektury, implementacji i badania empirycznego. AlphaPulse łączy pobieranie danych, walidowany schemat YAML, routing cech, modele, HPO, ocenę czasową, zapis proveniencji i eksport funkcji predykcyjnej. Benchmark pełnoskalowy potwierdził wykonalność kompletnego przepływu, a seria v5.3 pokazała, że framework potrafi porównywać heterogeniczne konfiguracje według jednego, domenowo poprawnego protokołu.

Wartość rozwiązania nie sprowadza się do pojedynczego estymatora. Kluczowym produktem jest kontrolowane środowisko eksperymentalne: konfiguracja opisuje zamiar badacza, warstwa wykonawcza realizuje ten sam podział i scoring, a baza prób wiąże wynik z kodem, danymi i ziarnem. Takie rozdzielenie odpowiada założeniom współczesnego AutoML, jednocześnie dodając wymagania specyficzne dla danych finansowych [10, 8, 9].

7.2 Interpretacja wyników modelowych

LightGBM i CatBoost osiągnęły najwyższe mediany CORR Sharpe na holdoutcie. Jest to spójne z pozycją gradient boostingu jako silnej klasy modeli dla danych tabelarycznych [12, 13]. Nie oznacza jednak uniwersalnej przewagi tych algorytmów. Przestrzeń była próbkowana adaptacyjnie, a modele fundamentowe działały z ostrzejszymi limitami pamięciowymi. Poprawna interpretacja brzmi zatem: w przyjętym budżecie i dla celu `target_ender_60` najlepsze konfiguracje pochodziły z dwóch rodzin boostingowych.

Umiarkowana korelacja rankingów holdout-validation potwierdza znaczenie separacji czasowej. W danych o niskim stosunku sygnału do szumu wybór najlepszego punktu na jednym przedziale łatwo przejmuje jego lokalną specyfikę. Purging ogranicza przeciek wynikający z nakładania się horyzontu celu, ale nie eliminuje niestacjonarności rynku [7]. Zewnętrzny przedział 88 er pełni dlatego inną

rolę niż holdout HPO: nie steruje samplerem i pozwala ocenić transfer wybranych receptur.

Najbardziej praktycznym wynikiem jest front Pareto. Próba 22 maksymalizuje CORR na validation przy dodatnim MMC, natomiast próba 4 maksymalizuje MMC przy nieco słabszym CORR. Są to dwa uzasadnione cele wdrożeniowe. Pierwszy profil preferuje stabilną zgodność z celem, drugi większy wkład niezależny od Meta Modelu [3, 4]. Zachowanie obu kandydatów jest bardziej informacyjne niż arbitralne połączenie metryk w jeden wskaźnik.

7.3 Odpowiedzi na pytania eksperymentalne

1. **Skalowalność i reprodukowalność.** Pełnoskalowy potok przetworzył 2,75 mln wierszy uczących i 3,90 mln wierszy walidacyjnych w 31,5 min. Każde uruchomienie zapisuje konfigurację, hash, metryki i artefakty, a końcowe HPO dodatkowo utrwała skróty danych oraz wersję kodu.
2. **Rodziny modeli.** W badanym budżecie najwyższe mediany holdoutu uzyskały CatBoost i LightGBM. Wynik ma charakter opisowy, ponieważ HPO nie jest zrównoważoną ablacją.
3. **Transfer i kompromis CORR–MMC.** Zgodność dwóch przedziałów jest dodatnia, ale umiarkowana. Próby 22 i 4 tworzą front Pareto na niezależnym validation i reprezentują odpowiednio profil CORR oraz MMC.
4. **Koszt i kompletność procesu.** Analiza obejmuje 27 kompletnych prób z medianą 127,2 s. Baza TrialDB zachowała 84,4% uruchomionej puli jako kompletne rekordy możliwe do bezpośredniego porównania.

Odpowiedzi te odnoszą się bezpośrednio do pomiarów dostępnych w artefaktach. Nie przypisują systemowi redukcji czasu pracy człowieka ani przewagi AutoResearch, ponieważ takie twierdzenia wymagałyby osobnego eksperymentu porównawczego.

7.4 Znaczenie inżynierskie

Projekt pokazuje, że wymagania domenowe można zamknąć w kontraktach oprogramowania. Kolumna ery nie jest zwykłą cechą, lecz steruje podziałem; horyzont celu wyznacza separację; oficjalne metryki są częścią ewaluatora; a format submisji jest sprawdzany w czystym podprocesie. Dzięki temu ryzyka metodologiczne są ograniczane przez architekturę, a nie tylko przez dyscyplinę osoby prowadzącej eksperyment.

Istotna jest również warstwa artefaktów. `extttprotocol.json` dokumentuje wejście i reguły badania, `exttttrials.db` przechowuje stan każdej próby, a W&B umożliwia porównanie krzywych oraz diagnostyk. Razem tworzą ślad audytowy potrzebny zarówno w pracy naukowej, jak i w iteracyjnym utrzymaniu modelu. Z kolei eksport `predict.pkl` zamyka lukę między eksperymentem i inferencją live.

7.5 Ograniczenia

Najważniejszym ograniczeniem jest budżet końcowej serii: jedno ziarno, 27 kompletnych konfiguracji i dziesięcioprocentowa próba danych uczących. Nie pozwala to oszacować wariancji między seedami ani przedstawić przedziałów ufności dla rankingu. Adaptacyjne próbkowanie TPE utrudnia także przyczynową interpretację różnic między rodzinami.

Drugie ograniczenie dotyczy selekcji czasowej. Holdout uczestniczy w funkcji celu, więc jego maksimum jest obciążone wielokrotnym porównaniem. Niezależny validation ogranicza to ryzyko, lecz 88 er nadal odpowiada jednemu okresowi rynkowemu. Wyniki nie są prognozą rezultatu live ani podstawą do automatycznego stakingu.

Benchmark pełnoskalowy używa innej wersji danych i celu niż końcowe HPO. Dostarcza dowodu skalowalności, ale nie stanowi baseline jakości. W pracy celowo nie ekstrapolowano wartości CORR lub MMC z próbki 10% na pełny zbiór: zależność jakości od liczby wierszy nie musi być liniowa.

7.6 Rekomendacja wdrożeniowa i dalszy rozwój

Pierwszym kandydatem do ponownego treningu na pełnych danych v5.3 jest próba 22, ponieważ łączy najwyższy CORR na niezależnym validation z dodatnim MMC. Próba 4 powinna zostać zachowana jako alternatywa nastawiona na unikalność sygnału. Po treningu pełnoskalowym oba modele należy oceniać w kolejnych rundach bez zmiany konfiguracji; dopiero taki monitoring pozwoli oszacować stabilność live.

Dalszy rozwój powinien objąć powtórzenia dla wielu ziaren, kroczące okna obejmujące więcej reżimów rynkowych oraz kontrolowane ablacje routingu cech, selekcji wariancji i neutralizacji. AutoResearch jest gotowym rozszerzeniem warstwy automatyzacji, ale jego wartość należy mierzyć przy identycznym budżecie i tej samej przestrzeni co TPE. Naturalnym krokiem produkcyjnym jest także automatyczny harmonogram retreningu, walidacji artefaktu i monitorowania dryfu metryk per-era.

7.7 Wnioski końcowe

AlphaPulse realizuje pełny, reprodukowalny cykl eksperymentu dla Numerai: od danych i deklaratywnej konfiguracji, przez czasowo poprawną walidację i HPO, po diagnostykę i eksport. Empirycznie system obsłużył trening pełnoskalowy oraz wyłonił na danych v5.3 dwa niezdominowane profile modelu. Najbardziej zrównowazona próba 22 osiągnęła CORR Sharpe 0,4247 i MMC Sharpe 0,2467 na niezależnym validation, a próba 4 podniosła MMC Sharpe do 0,3063.

Najważniejszym wkładem pracy jest połączenie poprawnej metodyki finansowej z inżynierią eksperymentów. Wynik nie jest pojedynczą liczbą, lecz systemem, który umożliwia odtwarzanie decyzji, uczciwe porównywanie konfiguracji i przejście od hipotezy do gotowego artefaktu predykcyjnego. W tym sensie AlphaPulse spełnia cel konfigurowalnego środowiska AutoML dla finansowych potoków Numerai.

Bibliografia

- [1] Palamabron, *AlphaPulse v0.6.0*. <https://github.com/Palamabron/AlphaPulse>.
- [2] Numerai, *Scoring — Numerai Tournament Documentation*. <https://docs.numer.ai/numerai-tournament/scoring/>, [dostęp: 23.08.2026].
- [3] Numerai, *Correlation (CORR) — Numerai Tournament Documentation*. <https://docs.numer.ai/numerai-tournament/scoring/correlation-corr>, [dostęp: 23.08.2026].
- [4] Numerai, *Meta Model Contribution (MMC) — Numerai Tournament Documentation*. <https://docs.numer.ai/numerai-tournament/scoring/meta-model-contribution-mmcc>, [dostęp: 23.08.2026].
- [5] Numerai, *Staking — Numerai Tournament Documentation*. <https://docs.numer.ai/numerai-tournament/staking>, [dostęp: 23.08.2026].
- [6] Numerai, *numerai-tools 0.6.0*, Python Package Index, 10.06.2026. <https://pypi.org/project/numerai-tools/0.6.0/>, [dostęp: 23.08.2026].
- [7] M. López de Prado, *Advances in Financial Machine Learning*. Wiley, 2018.
- [8] T. Akiba et al., “Optuna: A Next-generation Hyperparameter Optimization Framework”, *Proceedings of KDD*, 2019.
- [9] R. Liaw et al., “Tune: A Research Platform for Distributed Model Selection and Training”, *arXiv:1807.05118*, 2018.
- [10] X. He, K. Zhao and X. Chu, “AutoML: A Survey of the State-of-the-Art”, *Knowledge-Based Systems*, vol. 212, 2021.
- [11] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System”, *Proceedings of KDD*, 2016.
- [12] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”, *Advances in Neural Information Processing Systems*, 2017.
- [13] L. Prokhorenkova et al., “CatBoost: Unbiased Boosting with Categorical Features”, *Advances in Neural Information Processing Systems*, 2018.
- [14] D. H. Wolpert, “Stacked Generalization”, *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [15] N. Hollmann et al., “Accurate Predictions on Small Data with a Tabular Foundation Model”, *Nature*, vol. 637, pp. 319–326, 2025.